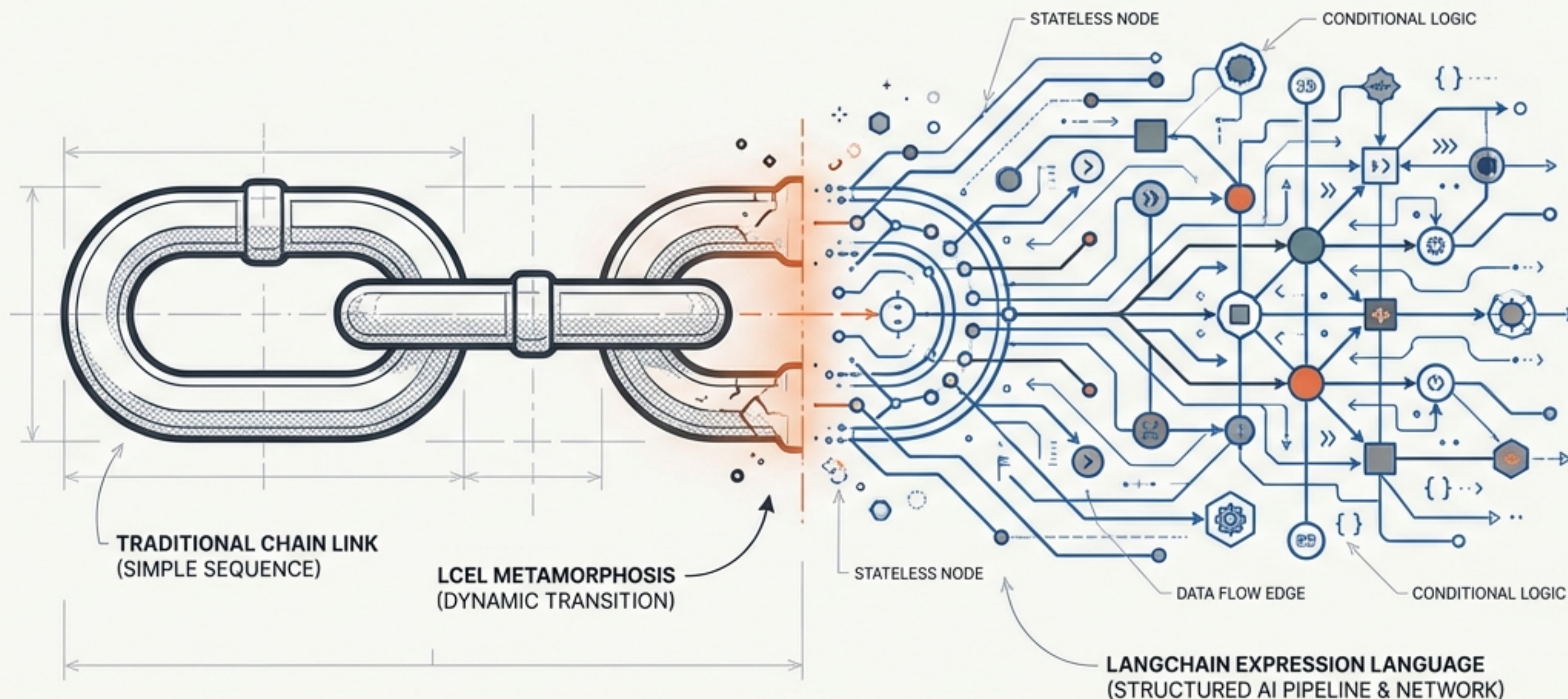


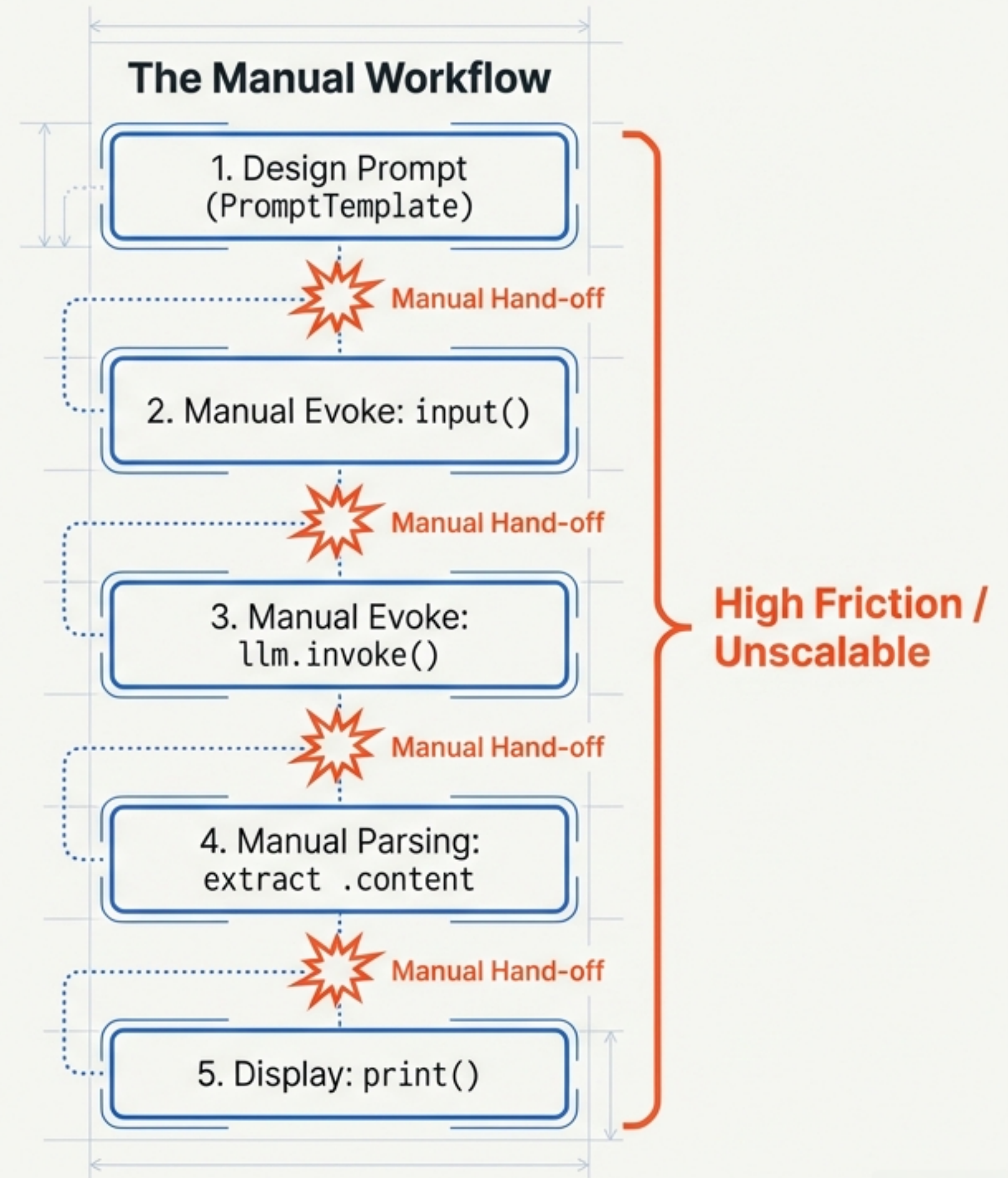
MASTERING CHAINS IN LANGCHAIN

Building Automated, Scalable Generative AI Pipelines



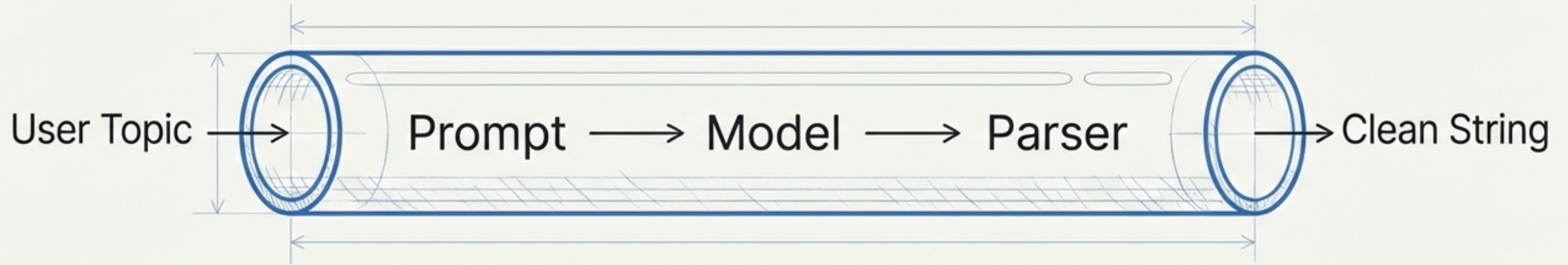
The “Manual Trap” of Building AI Apps

Building applications by manually executing individual steps is brittle and unscalable. As complexity grows, the manual hand-off between components creates friction.



Enter the Chain: The Pipeline Paradigm

Trigger once, execute all. The output of Step 1 automatically becomes the input of Step 2.



The Old Way (Nested Calls)

```
model.invoke(prompt.invoke(topic))
```

The New Way (LCEL Syntax)

```
# The Pipe Operator (|) connects components  
chain = prompt | model | parser  
chain.invoke("Cricket")
```

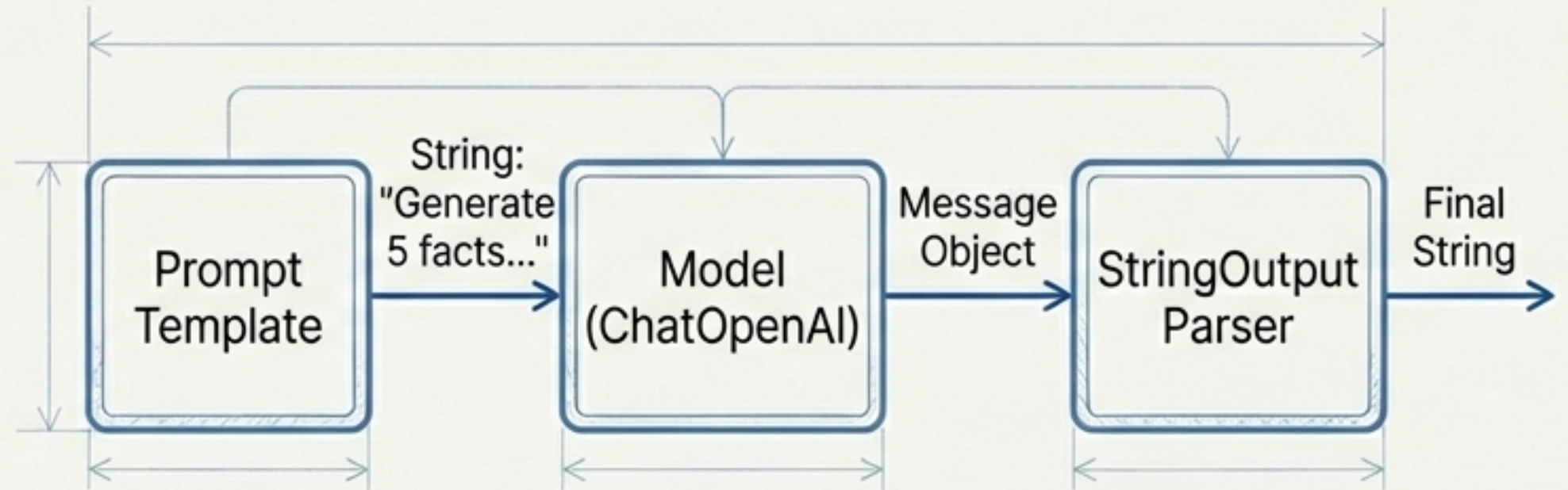

Pattern 1: The Sequential Chain

The fundamental linear workflow: $A \rightarrow B \rightarrow C$

Use Case: Fact Generator

1. Define the steps
`chain = prompt | model | parser`

2. Execute
`result = chain.invoke("Cricket")`



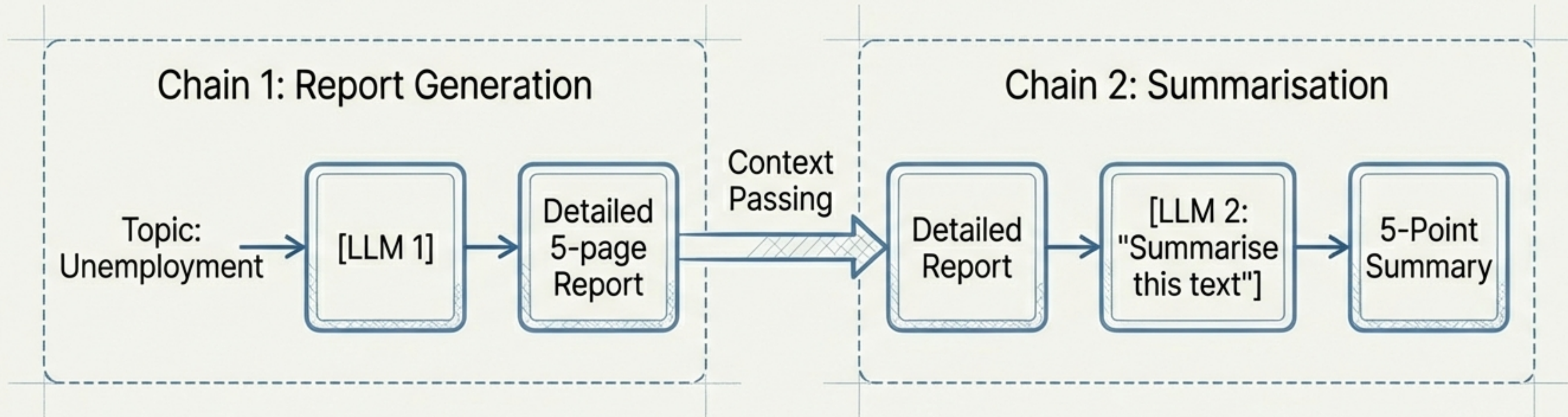
JetBrains Mono

```
1. Cricket is the second most popular sport...
2. The longest match lasted 9 days...
3. ...
```


Extended Sequences: Chaining the Chains

How do you maintain context across multiple LLM calls?

By using the text output of the first chain as the variable input for the second.



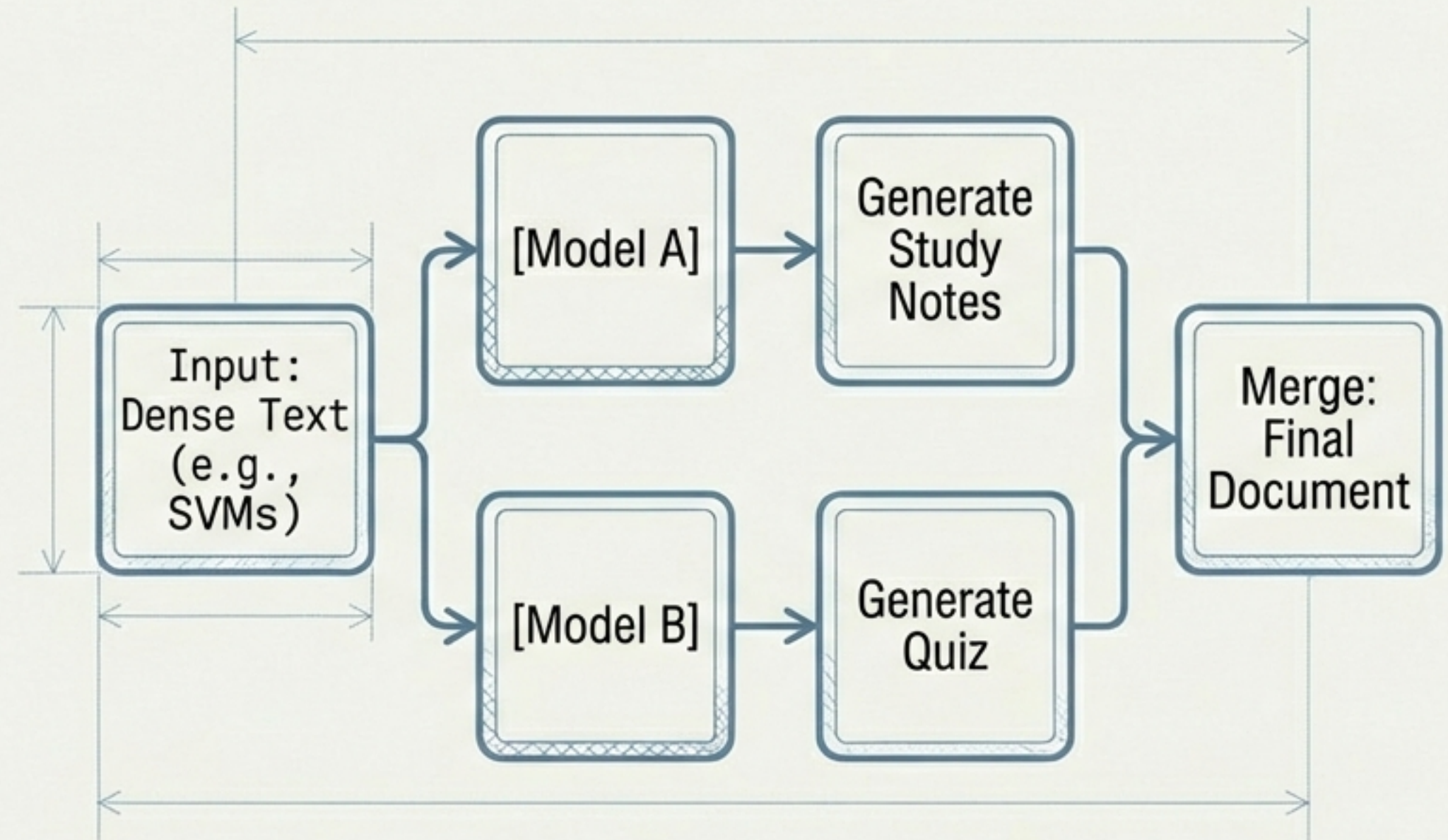
No manual intervention required between Stage 1 and Stage 2.

Pattern 2: Parallel Chains

Simultaneous execution for multi-faceted outputs.

Why wait for one task to finish before starting another? Use parallel chains to split a single input into multiple processing branches.

Example Use Case:
Educational Tool. Input a technical document and generate Study Notes AND a Quiz at the same time.



Implementing Parallelism with RunnableParallel

The `RunnableParallel` primitive allows you to define a dictionary map where keys are branch names and values are chains.

```
from langchain_core.runnables import RunnableParallel
```

```
# Define the branches in a dictionary map
```

```
map_chain = RunnableParallel({  
    "notes": prompt1 | model1 | parser,  
    "quiz": prompt2 | model2 | parser  
})
```

```
# The final chain merges the map results
```

```
chain = map_chain | merge_chain
```

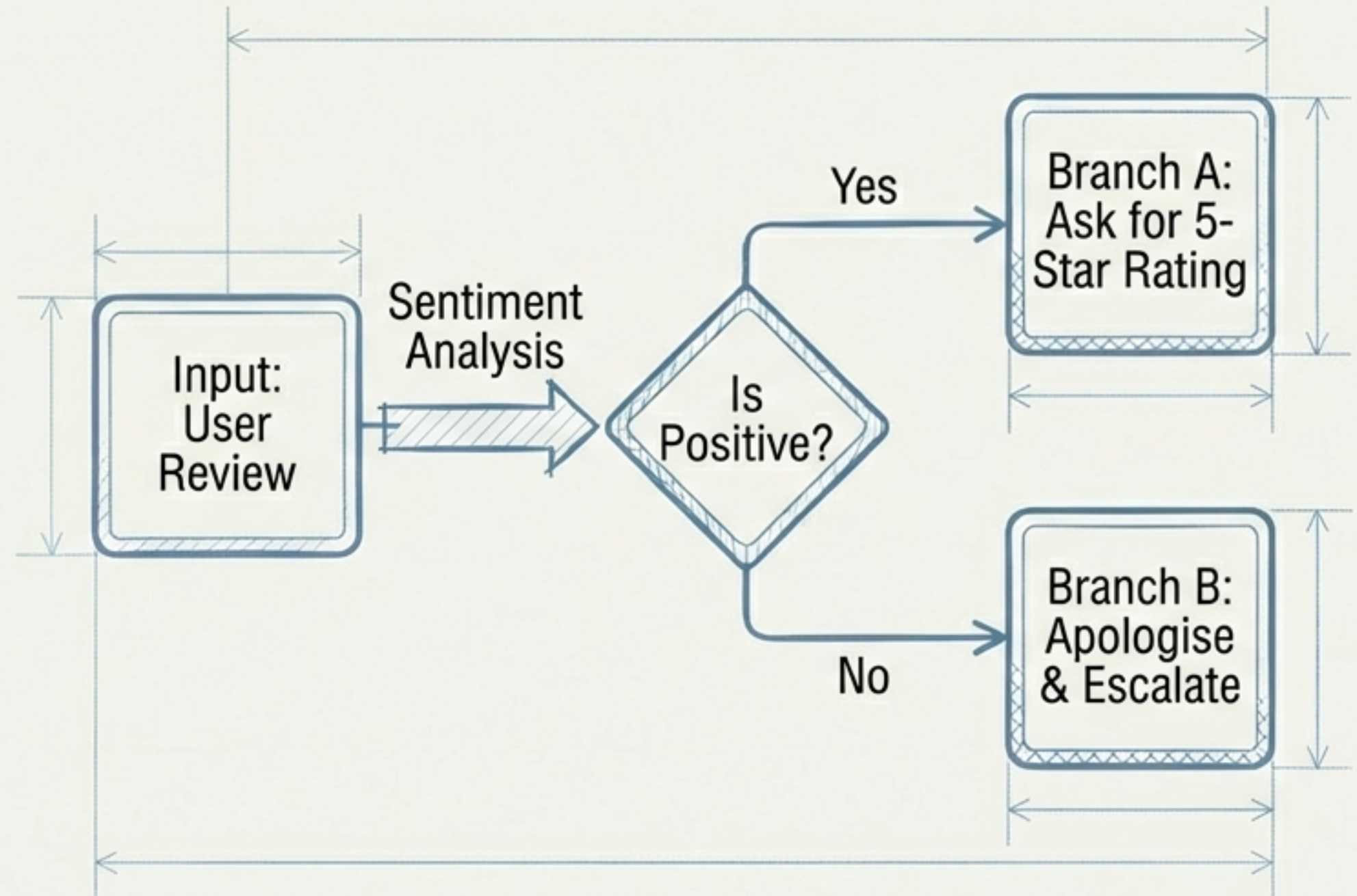
These keys become the
input variables for the
merge_chain

Pattern 3: Conditional Chains

Injecting decision-making logic into the pipeline.

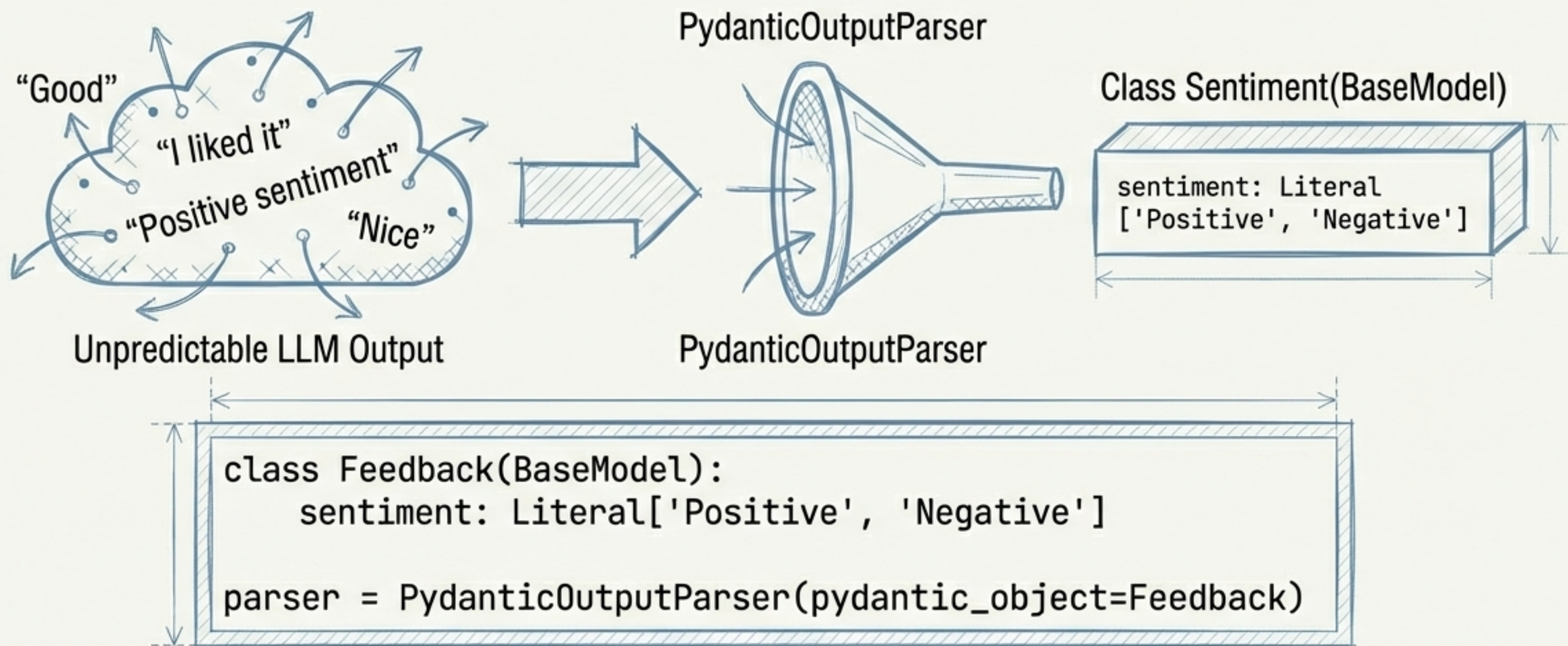
The chain adapts based on the content it processes. This introduces If/Else logic: only one path is executed based on a condition.

Use Case:
Customer Feedback Loop



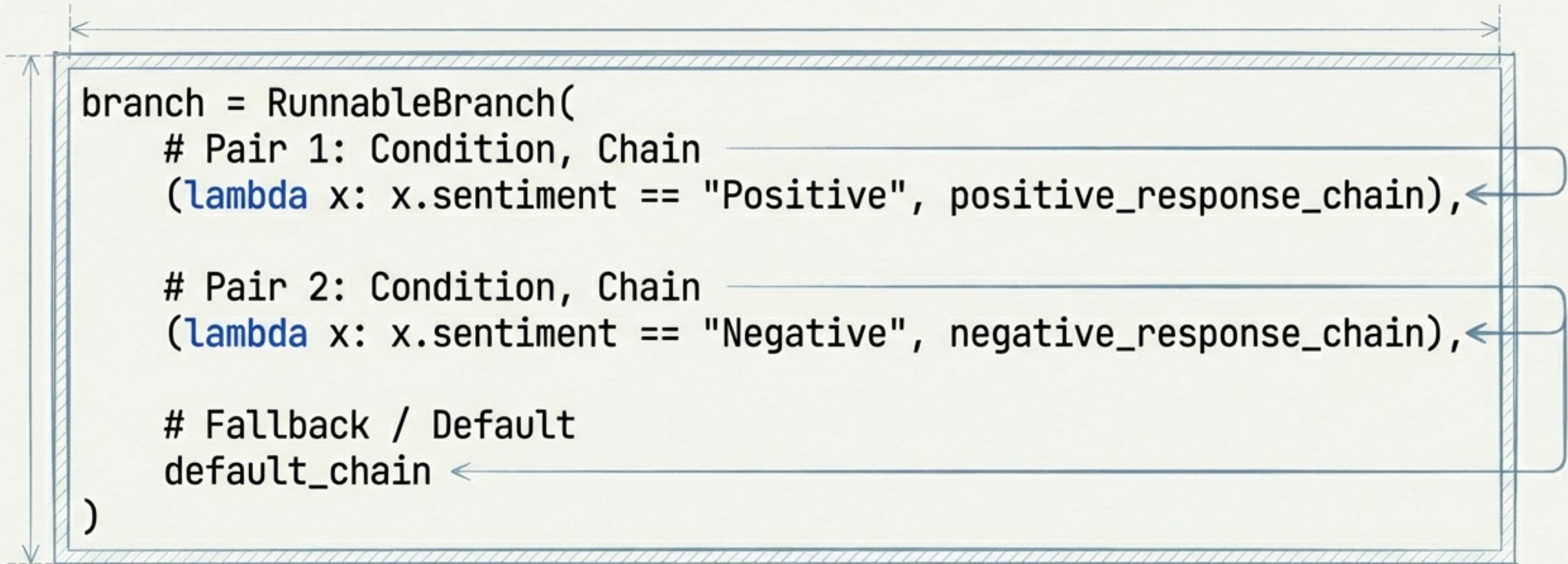
The Prerequisite: Structured Output

You cannot build reliable logic (`if x == 'Positive'`) on unstructured free text. We must enforce strict categorical output.



Routing Logic with RunnableBranch

The Router defines a list of (Condition, Chain) tuples. The first condition to return True triggers its chain.



```
branch = RunnableBranch(  
    # Pair 1: Condition, Chain  
    (lambda x: x.sentiment == "Positive", positive_response_chain),  
  
    # Pair 2: Condition, Chain  
    (lambda x: x.sentiment == "Negative", negative_response_chain),  
  
    # Fallback / Default  
    default_chain  
)
```

The diagram illustrates the structure of the `RunnableBranch` function. A large blue double-lined rectangle encloses the entire code block. A horizontal double-headed arrow at the top spans the width of the code. A vertical double-headed arrow on the left spans the height of the code. Three horizontal arrows point from the right side of the code to the right, each corresponding to one of the three argument pairs: the first pair (lambda condition, positive_response_chain), the second pair (lambda condition, negative_response_chain), and the third pair (default_chain). These arrows indicate that each pair represents a condition-chain tuple.

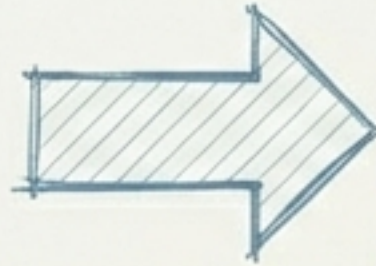
Handling Edge Cases: The Default Branch

Ensuring pipeline integrity with RunnableLambda.

Standard Python functions are not 'Runnables' and cannot be directly plugged into a chain. To create a valid default fallback step, wrap your function in 'RunnableLambda'.



```
def fallback(x):  
    return "Error"
```



```
RunnableLambda(fallback)
```

```
from langchain_core.runnables import RunnableLambda
```

```
def fallback(x):  
    return "Could not classify sentiment"
```

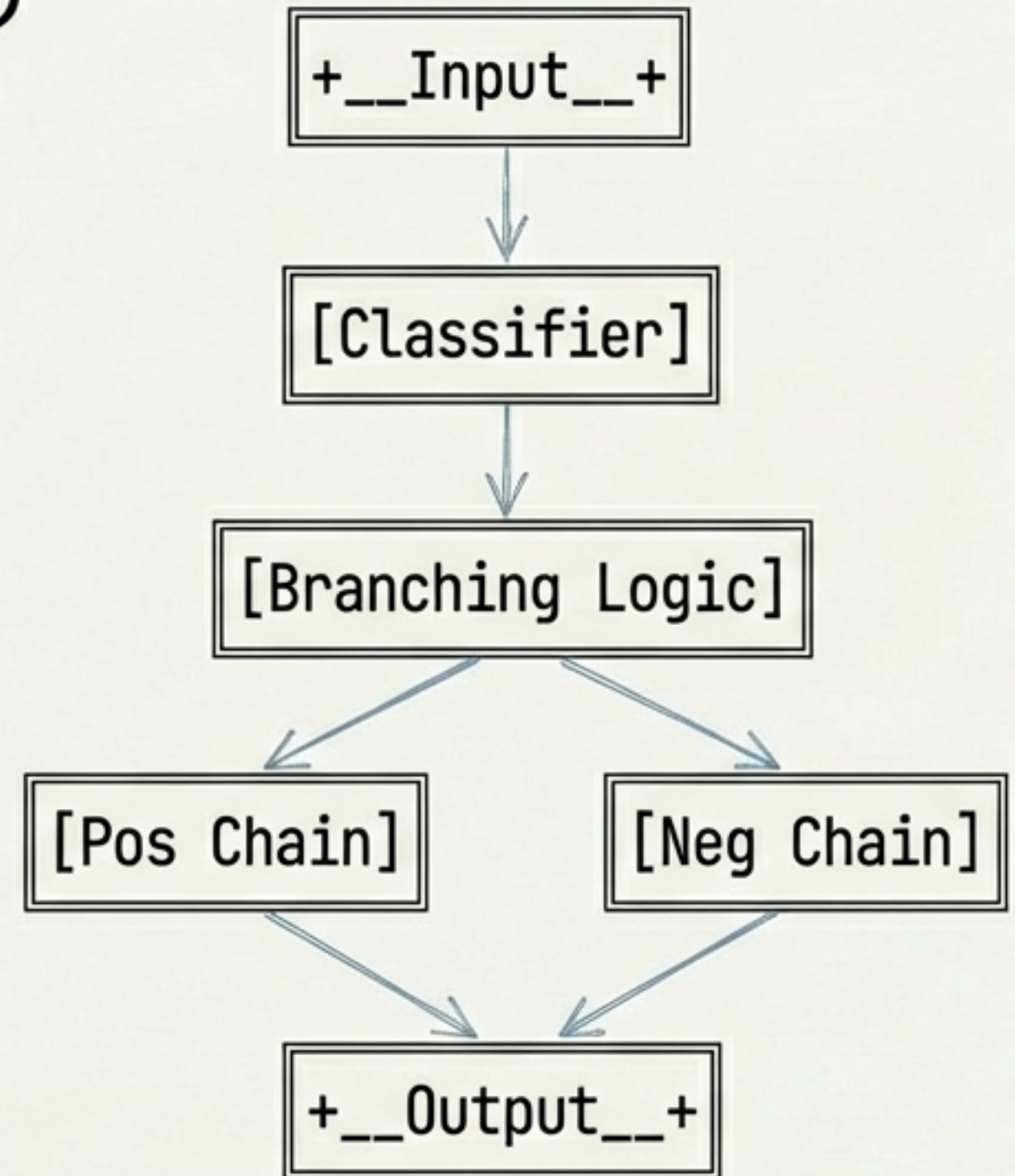
```
# Wrap it to make it compatible  
default_chain = RunnableLambda(fallback)
```


Visualising the Architecture

Debugging complex flows with ``get_graph()``

LangChain is not a black box. You can inspect the actual graph structure of your defined chain to verify logic paths.

```
chain.get_graph().print_ascii()
```



Architecture Recap

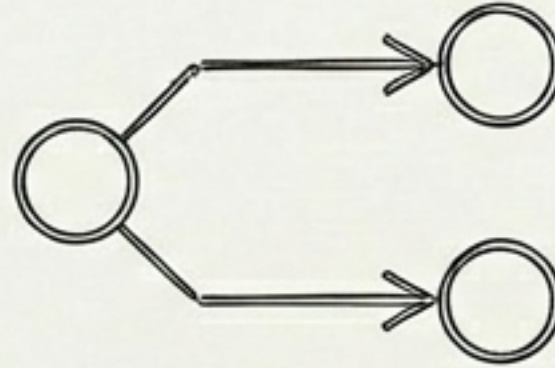
Sequential



A | B | C

Use: Linear tasks where output of A is input of B.

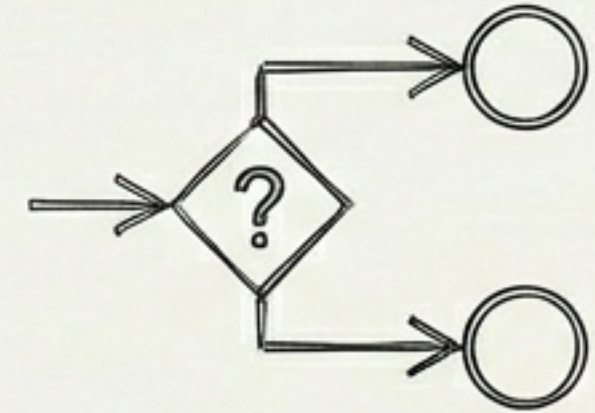
Parallel



`RunnableParallel({
 {key: chain}`

Use: Efficiency and merging distinct perspectives.

Conditional

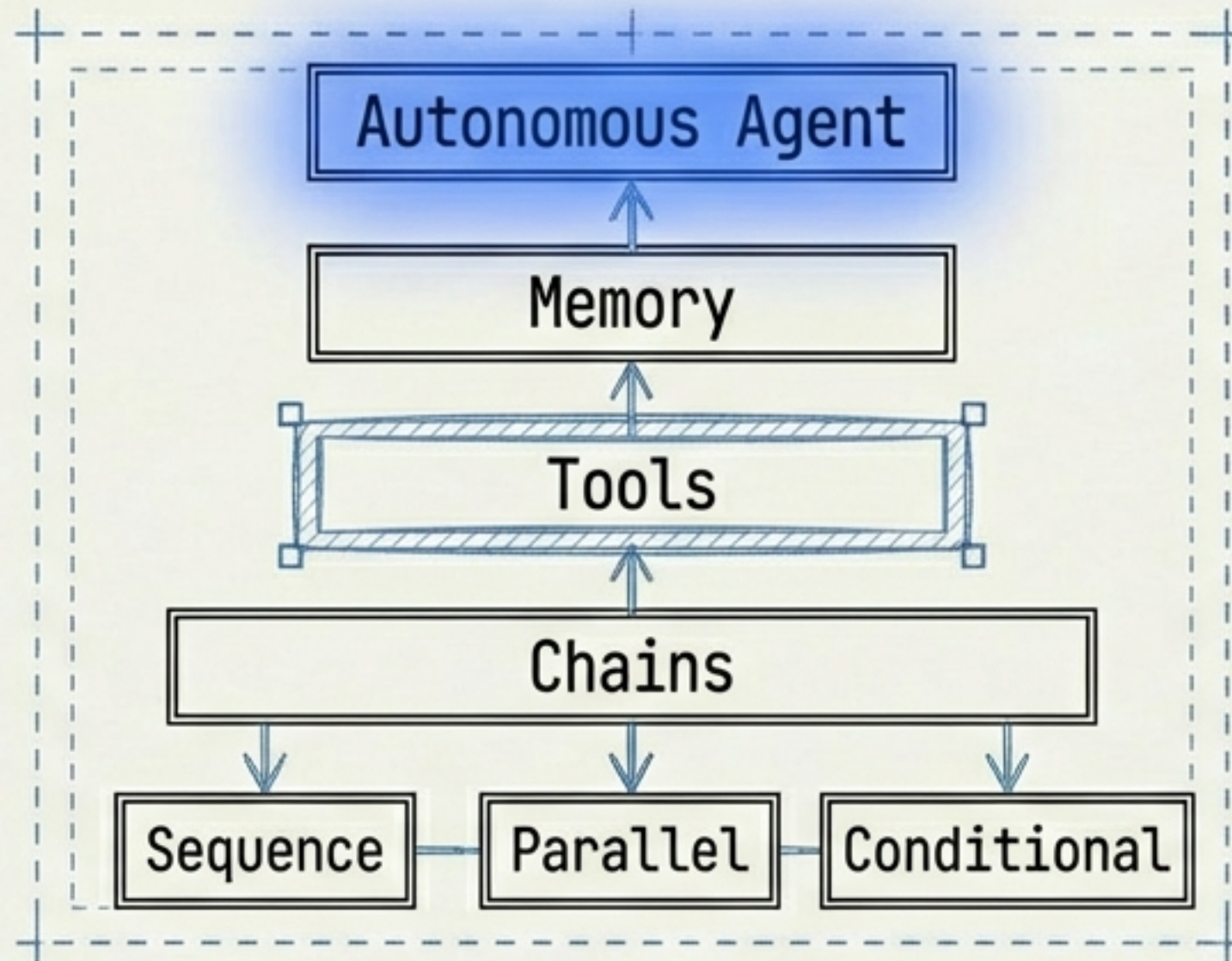


`RunnableBranch(
 ((condition, chain))`

Use: Logic and decision making (requires structured output).

The Foundation of Agents

The patterns we covered—Sequence, Parallelism, and Conditions—are the primitive building blocks of autonomous Agents.



Next Up: Runnables (The magic behind the scenes)

Resources & Credits

Content based on the 'Generative AI' series by CampusX.
Code samples adapted for LangChain v0.1+

